

Python

```
import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_curve
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Check tensorflow and sklearn versions
print(f"TF Version: {tf.__version__}")
```

Code output

TF Version: 2.20.0

This Proof of Concept (POC) is designed to detect fraudulent credit card transactions by combining **Unsupervised Anomaly Detection** and **Supervised Deep Learning**.

Part 1: The Fraud Detection Notebook

You can copy the code below into a Jupyter Notebook ([.ipynb](#)) or a Google Colab instance.

1. Data Ingestion & Preprocessing

This script loads the dataset directly from a public repository. We scale the features and split the data, keeping the original class imbalance to simulate a real-world scenario.

Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import IsolationForest
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_curve, f1_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
# 1. Load Data (Kaggle Credit Card Fraud Mirror)
url =
"https://raw.githubusercontent.com/nsethi31/Kaggle-Data-Credit-Card-Fraud-Detection/master/credit-card.csv"
df = pd.read_csv(url)

# 2. Preprocessing
# Scale 'Amount' and 'Time' as V1-V28 are already PCA-transformed
scaler = StandardScaler()
df['Amount'] = scaler.fit_transform(df['Amount'].values.reshape(-1, 1))
df['Time'] = scaler.fit_transform(df['Time'].values.reshape(-1, 1))

# Split into Features and Target
X = df.drop('Class', axis=1)
y = df['Class']

# Split for supervised training (Deep Learning)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

2. Unsupervised Learning: Isolation Forest

Isolation Forest is ideal for fraud because it identifies "anomalies" (outliers) rather than learning "normal" patterns.

Python

```
# Train Isolation Forest
# contamination is the approximate percentage of fraud (0.17%)
iso_forest = IsolationForest(n_estimators=100, contamination=0.0017, random_state=42)
iso_preds = iso_forest.fit_predict(X_test)

# Map predictions: Isolation Forest returns -1 for outliers and 1 for inliers
iso_preds = [1 if x == -1 else 0 for x in iso_preds]

print("--- Isolation Forest Evaluation ---")
print(classification_report(y_test, iso_preds))
```

3. Deep Learning: Neural Network

We use a simple Dense Neural Network (DNN) with Dropout layers to prevent overfitting on the majority class.

Python

```

# Build DNN Model
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train (using class weights to handle imbalance)
class_weights = {0: 1, 1: 50} # Give more weight to fraud cases
history = model.fit(X_train, y_train, epochs=10, batch_size=2048, validation_split=0.1,
                    class_weight=class_weights, verbose=0)

# Predictions (Threshold 0.5)
dl_probs = model.predict(X_test)
dl_preds = (dl_probs > 0.5).astype(int)

print("--- Deep Learning Evaluation ---")
print(classification_report(y_test, dl_preds))

```

4. Performance Visualization

We prioritize the **Precision-Recall Curve** over Accuracy, as it is more meaningful for imbalanced datasets.

Python

```

# Plot Confusion Matrix and PR Curve
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

# Confusion Matrix
sns.heatmap(confusion_matrix(y_test, dl_preds), annot=True, fmt='d', cmap='Blues', ax=ax[0])
ax[0].set_title('Deep Learning Confusion Matrix')
ax[0].set_xlabel('Predicted')
ax[0].set_ylabel('Actual')

# Precision-Recall Curve
precision, recall, _ = precision_recall_curve(y_test, dl_probs)
ax[1].plot(recall, precision, color='purple', lw=2)
ax[1].set_title('Precision-Recall Curve')
ax[1].set_xlabel('Recall')
ax[1].set_ylabel('Precision')

plt.show()

```

Part 2: Deployment Strategy

Deploying a fraud model requires high availability and real-time processing. Below is a multi-cloud component map and the architectural flow.

Cloud Components Table

Service Type	AWS	Azure	GCP
Model Hosting	SageMaker Real-time Endpoints	Azure ML Online Endpoints	Vertex AI Prediction
Stream Processing	Amazon Kinesis	Azure Event Hubs	Google Cloud Pub/Sub
Serverless Logic	AWS Lambda	Azure Functions	Cloud Functions
Monitoring	SageMaker Model Monitor	Azure Monitor	Vertex AI Model Monitoring

MLOps Architecture (Mermaid)

Code snippet

```
graph LR
  subgraph "Data & Streaming"
    A[Transaction Source] --> B[Kinesis / Pub-Sub / Event Hubs]
  end

  subgraph "Real-Time Inference"
    B --> C[Serverless Function]
    C --> D["Model Endpoint: SageMaker/Vertex/AzureML"]
    D --> E[Decision Engine]
  end
  End
```

```
subgraph "Storage & Feedback"
  E -->|Approved| F[Transaction DB]
  E -->|Flagged| G[Fraud Analyst UI]
  G -->|Feedback| H[Feature Store]
  H -->|Retrain| D
end
```

